LISRec 詳解

LISRec: Modeling User Preferences with Learned Item Shortcuts for Sequential Recommendation
KDD 2026 · 東北大學、清華大學、阿里巴巴、CMU · 程式碼：github.com/NEUIR/LISRec

一、一句話總結

使用者的點擊紀錄裡有不少隨手亂點的雜訊。LISRec 先用語言模型算出物品之間的語意相似度，再為每個使用者建一張「物品相似度圖」，只保留圖中**最大的連通分量**當作這個使用者的代表性歷史。論文稱這個子集為「捷徑」（shortcut），推薦模型只用這個子集來學使用者的喜好。

二、要解決的問題

序列推薦（Sequential Recommendation）的任務是：給定使用者依時間排序的互動歷史 $H = \{v_1, \ldots, v_{t-1}\}$，預測下一個會互動的物品 v_t。

作者指出，真實的互動序列裡有**雜訊**。論文 Figure 1 的例子是：一個使用者一直買 LEGO（鋼鐵人、Stitch、Bugatti、星際大戰 R2-D2），中間卻點了一次 Nintendo Switch 的瑪利歐賽車，可能只是一時好奇。

現有方法的問題：

類型	代表方法	問題
物品 ID 模型	GRU4Rec、SASRec、BERT4Rec	每筆互動都同等看待，雜訊會被一起學進使用者表示
文字內容模型	TASTE、HLLM	同上，所有物品都餵進去
隱式去雜訊	調整樣本權重、對比學習（DCRec）	沒有明確把雜訊拿掉，難以解釋
顯式去雜訊	DSAN、HSD	大多依賴物品 ID 表示，需要大量互動資料，沒用到文字語意

作者的切入點：**用文字語意來判斷哪些互動「彼此一致」，把不一致的離群點明確拿掉。**

三、整體架構

整個系統有兩個模組（對照論文 Figure 2）：

1. **捷徑萃取器 M_{Filter}**：從完整歷史 H 中挑出子集 $H_{Shortcut} \subseteq H$
2. **推薦模組 M_{Rec}**：只用 $H_{Shortcut}$ 建模使用者並做推薦

```
完整歷史 H ──M_Filter──▶ H_Shortcut ──M_Rec──▶ 預測下一個物品
```

3.1 推薦模組 M_{Rec}（沿用 TASTE）

物品表示：把物品的 ID 和屬性轉成文字模板，再用 T5 編碼：

```
X(v) = "id: v(id) … <Attr>k: v(<Attr>k)"
v⃗ = T5(X(v))
```

使用者表示：把捷徑裡的物品文字串接起來，再用 T5 編碼：

```
X(H_Shortcut) = ‖ X(vi)，vi ∈ H_Shortcut
H⃗_Shortcut = T5(X(H_Shortcut))
```

這裡要注意，捷徑裡的物品**不一定連續，也不保證原本的順序**。

預測：使用者向量和候選物品向量做內積，再對所有物品取 softmax：

```
P(v | H) = Softmax_v(H⃗ · v⃗)
```

訓練時用 in-batch negatives。

3.2 捷徑萃取器 M_{Filter}（核心貢獻）

分三步。

步驟 1：與任務無關的語意物品編碼（Task-Agnostic Semantic Item Encoding）

目的是學到「跟使用者喜好有關」的物品相似度，而不是單純的文字相似度。

- **只用文字屬性、不用物品 ID**：$\tilde{X}(v)'$ = "<Attr>1: … <Attr>k: …"。這樣比較能泛化，也不依賴稀疏的 ID。
- **用 Next Item Prediction 做 instruction tuning**：把歷史串成文字，加上指令，讓 T5 預測下一個物品：

  > "This is the T dataset. Here is the visit history list of the user: $\tilde{X}'(H)$, recommend next item"

- **跨資料集混合訓練**：把 Beauty、Sports、Toys、Yelp 的資料重新取樣，並平衡各資料集的數量（附錄 Table 7：訓練資料約 90 萬筆，驗證資料約 7.8 萬筆），避免偏向單一資料集。

訓練完之後，兩個物品的相似度用 cosine 計算：

```
Sij = cos(v⃗i, v⃗j)
```

步驟 2：建立個人化物品圖

對每個使用者 U，用他互動過的所有物品建一張無向圖 $G_U = (V, E)$：

```
eij = 1   若 Sij ≥ τ
eij = 0   若 Sij < τ
```

論文設定 $\tau = 0.7$。語意差太多的物品就不會連邊。

步驟 3：取最大連通分量當捷徑

```
C_max = argmax_{Ci} |V(Ci)|
H_Shortcut = { vj | vj ∈ H 且 vj ∈ V(C_max) }
```

最大連通分量的直覺是：使用者「主要的、一致的喜好」應該是一大群彼此相似的物品，雜訊則是孤立點或很小的群。

推論細節：建圖時會排除最後一筆互動，避免資料洩漏。

四、理論分析（附錄 A.3）

作者證明，在三個假設下，最大連通分量**正好等於**使用者的真實喜好集合 P：

1. **喜好節點彼此可達**：P 裡的任兩個物品，都能透過相似度 $\geq \tau$ 的邊連到。
2. **雜訊和喜好之間沒有邊**：任何雜訊物品和喜好物品的相似度都 $< \tau$。
3. **雜訊群夠小**：雜訊就算彼此相似、聚成小群，大小也一定小於 $|P|$。

推導很直接：由 (1)(2)，P 自成一個連通分量；由 (3)，其他分量都比它小，所以 $C_{max} = P$。

> **評論**：這三個假設其實已經很接近結論本身，證明主要是把直覺形式化。實際上最可能出問題的是假設 (2)（雜訊和喜好物品偶爾會相似）和假設 (3)（使用者可能有好幾個同樣重要的興趣）。

五、實驗設定

- **資料集**：Amazon Beauty、Sports、Toys，以及 Yelp
- **前處理**：用 RecBole，過濾互動少於 5 次的使用者和物品；用 leave-one-out 切分，最後一筆當測試、倒數第二筆當驗證
- **評估方式**：Recall@10/20、NDCG@10/20，**對全部物品排序**（full ranking，不是抽樣評估）；顯著性用 permutation test，$p < 0.05$
- **Baseline**：
  - ID 模型：GRU4Rec、BERT4Rec、SASRec、S3Rec、NOVA、ICAI-SR、DIF-SR
  - 文字模型：CoWPiRec、DWSRec、HLLM（用 1.1B 的 TinyLlama 重現）、TASTE
  - 去雜訊方法：DSAN、HSD、DCRec（都以 BERT4Rec 為基底）
- **實作**：M_{Filter} 和 M_{Rec} 都從 T5-base 初始化，學習率 1e-4；M_{Filter} 的 batch size 32，M_{Rec} 的 batch size 8

六、實驗結果

6.1 主要結果（Table 1）

LISRec 以 TASTE 為基底，四個資料集大多數指標都是第一。例如 Beauty 的 R@10 是 0.1249，TASTE 是 0.1030。論文宣稱最多約 13% 的提升。

- **Beauty、Sports 進步最多**，Yelp 最少。作者的解釋是購物情境中隨手點擊比較多。
- 文字模型整體明顯優於 ID 模型。

6.2 去雜訊能力（Table 2）

把 LISRec 的過濾器接到 BERT4Rec 上，和 DSAN、HSD、DCRec 比較，四個資料集都是 LISRec 最好，平均好約 4.7%。這也說明**過濾器可以接在不同的推薦模型前面**。

6.3 消融實驗（Table 3、Table 4）

版本	說明	結果
w/o M_{Filter}	不過濾，等於原本的 TASTE	最差
Zero-Shot	用沒訓練過的 T5 過濾	已經比不過濾好
w/o M_{Rec} Training	過濾器有訓練，但推薦模型沒用過濾後的資料重新訓練	更好
完整 LISRec	兩者都訓練	最好，比 Zero-Shot 平均多 12% 以上

Table 4 再比較三種過濾器：原始 T5、用單一資料集訓練的 TASTE、LISRec 的跨資料集 instruction tuning。LISRec 最好，表示**跨資料集訓練**有幫助。

6.4 門檻 τ 的影響（Figure 3）

- τ 太低（0.2–0.4）：過濾太寬鬆，雜訊混進來
- τ 中等（0.5–0.7）：表現最好
- **任何 τ 都比不過濾好**，對超參數不太敏感

6.5 用哪種物品表示來過濾（Figure 4、5）

作者比較用 BERT4Rec（ID）、BGE-Base（通用檢索模型）、LISRec 的表示來建圖：

- LISRec 選出來的物品，**彼此相似度最高**，和**目標物品的相似度也最高**。
- 用這些物品訓練的推薦模型，Recall 也最好，在大的 cutoff 差距更明顯。

6.6 對推薦模組的影響（Figure 6）

- 用捷徑訓練後，**使用者表示和真實目標物品更接近**。
- t-SNE 視覺化顯示，被移除的物品在 embedding 空間中被推到外圍，成為離群點。

6.7 案例分析（Table 5）

Amazon Beauty：

物品	內容
n1	保濕乳液
n2	保濕乳霜
n3	Valentino 香水
n4	夜間保濕乳

- n1、n2、n4 兩兩相似度約 0.44–0.46，彼此連邊。
- n3 和其他物品只有 0.20–0.25，成為孤立點。
- 最後的捷徑是 {1, 2, 4}，香水被濾掉。

Yelp：使用者常去酒吧和餐廳，中間去過一次動物醫院。動物醫院和其他地點相似度低，所以被排除。

七、優點

1. **明確且可解釋**：可以直接看到哪些點擊被移除、為什麼被移除。
2. **隨插即用**：過濾器和推薦模型分開，TASTE 和 BERT4Rec 都能接。
3. **利用文字語意**：不依賴物品 ID，冷門物品也能判斷相似度。
4. **方法簡單**：核心只是「建圖加上找最大連通分量」，計算成本低。
5. **對 τ 穩健**：不同門檻都有進步。

八、限制與可以討論的地方

1. **只保留一個興趣**：最大連通分量只留下一群物品。如果使用者有兩個同樣真實的興趣（例如同時喜歡 LEGO 和電玩），比較小的那個會被誤判為雜訊。這正好違反理論分析的假設 (3)。
2. **丟掉時間順序**：捷徑裡的物品不一定連續，也沒有保證原本順序。對「序列」推薦來說，最近的興趣轉變可能被忽略。
3. **案例分析和設定不一致**：主實驗用 τ = 0.7，但 Table 5 的案例中，相似度只有 0.44 也連了邊。案例可能用了不同的門檻或相似度尺度，論文沒有說明。
4. **表格數字不完全一致**：
   - Sports 的 LISRec N@20，Table 1 是 0.0463，Table 3、Table 4 是 0.0496。
   - Toys 的 LISRec N@20，Table 1 是 0.0786，Table 3 是 0.0826。
5. **不是每個指標都第一**：
   - Toys 的 N@10：TASTE 0.0640 > LISRec 0.0637
   - Yelp 的 N@10：DIF-SR 0.0419 > LISRec 0.0408
6. **Yelp 進步很小**：在雜訊比較少的情境，效果有限。
7. **理論分析偏弱**：假設強到幾乎等於結論，對實際何時會失效的幫助不大。
8. **依賴文字品質**：物品描述太短或不準確時，相似度就不可靠。

九、和 Paper B（新聞推薦論文）的關係（報告時可以提）

	Paper A：LISRec（KDD '26）	Paper B：Stage-wise 新聞推薦（WWW '26）
挑戰的假設	歷史是**乾淨**的	歷史是**靜態**的
處理方式	依**語意**篩選歷史	依**時間**切開歷史
會丟掉什麼	丟掉離群的點擊	不丟資料，只是分段

兩者可以互補：先用 LISRec（Paper A）去掉雜訊，再用 Paper B 的方式建模興趣隨時間的變化。